

DAY 8: PYTHON EXCEPTION HANDLING + DEBUGGING

Data Science 60-Day Learning Journey

Student: Muhammad Ahmad

Status: Day 8 Completed

Focus: Errors, Exceptions, Validation, Debugging, CSV/JSON Error Handling, Safe Data Processing

1. Learning Objectives

Understand errors and exceptions, distinguish syntax and runtime errors, use try/except/else/finally, handle specific exceptions, raise exceptions for validation, create custom exceptions, debug Python programs, and safely work with files, CSV, and JSON.

2. Errors and Exceptions

An error is a problem that prevents a program from behaving as intended. Syntax errors are detected before normal execution. Runtime errors occur while the program is running and are commonly represented by exceptions.

NameError

```
print(name)
```

The variable was never defined, so Python raises NameError.

ZeroDivisionError

```
10 / 0
```

Python cannot divide by zero, so it raises ZeroDivisionError.

3. Syntax Errors

```
if 10 > 5  
    print("Yes")
```

The colon is missing. Correct code:

```
if 10 > 5:  
    print("Yes")
```

Common syntax problems include missing colons, brackets, quotation marks, incorrect indentation, and invalid Python syntax.

4. Runtime Errors

```
number = 10  
result = number / 0
```

This code has valid Python syntax, but it fails during execution.

5. try and except

The try block contains risky code. The except block handles an exception.

```
try:  
    number = 10 / 0  
except:
```

```
print("Something went wrong")
```

Prefer specific exceptions:

```
try:
    number = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")
```

6. Important Exceptions

ValueError

```
age = int("hello")

try:
    age = int("hello")
except ValueError:
    print("Please enter a valid number")
```

TypeError

```
name = "Ahmad"
age = 22
print(name + age)
```

Fix by converting the integer or printing separately:

```
print(name + str(age))
print(name, age)
```

NameError

```
try:
    print(student_name)
except NameError:
    print("Variable does not exist")
```

IndexError

```
students = ["Ahmad", "Ali"]
try:
    print(students[5])
except IndexError:
    print("Invalid list index")
```

KeyError

```
student = {"name": "Ahmad", "cgpa": 3.23}

try:
    print(student["age"])
except KeyError:
    print("Age key does not exist")
```

Safer dictionary access:

```
print(student.get("age"))
print(student.get("age", "Not available"))
```

FileNotFoundError

```
try:
    with open("students.txt", "r") as file:
        data = file.read()
except FileNotFoundError:
    print("File does not exist")
```

7. Multiple Exceptions

```

try:
    number = int(input("Enter number: "))
    result = 100 / number
except ValueError:
    print("Please enter a valid number")
except ZeroDivisionError:
    print("Number cannot be zero")

```

8. Exception as a Variable

```

try:
    number = 10 / 0
except ZeroDivisionError as error:
    print("Error:", error)

```

This lets you inspect the actual exception message.

9. else and finally

else runs only when try succeeds. finally runs whether an exception happens or not.

```

try:
    number = 10 / 2
except ZeroDivisionError:
    print("Cannot divide by zero")
else:
    print("Calculation successful")
finally:
    print("Program finished")

```

10. Complete Exception Handling Pattern

```

try:
    number = int(input("Enter number: "))
    result = 100 / number
except ValueError:
    print("Invalid number")
except ZeroDivisionError:
    print("Cannot divide by zero")
else:
    print("Result:", result)
finally:
    print("Program completed")

```

11. raise and Data Validation

raise lets you intentionally create an exception when a rule is violated.

```

age = -5
if age < 0:
    raise ValueError("Age cannot be negative")

```

For CGPA validation:

```

cgpa = 5.2
if cgpa < 0 or cgpa > 4:
    raise ValueError("CGPA must be between 0 and 4")

```

12. Custom Exceptions

```

class InvalidMarksError(Exception):
    pass

marks = 120
if marks > 100:
    raise InvalidMarksError("Marks cannot be greater than 100")

```

Custom exceptions make application-specific validation rules easier to understand.

13. Exception Handling with Functions

```
def divide(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        return "Cannot divide by zero"

print(divide(10, 2))
print(divide(10, 0))
```

14. Safe User Input

```
while True:
    try:
        number = int(input("Enter a number: "))
        print("You entered:", number)
        break
    except ValueError:
        print("Invalid input. Try again.")
```

15. Safe Calculator

```
def calculator():
    try:
        a = float(input("Enter first number: "))
        b = float(input("Enter second number: "))
        operation = input("Enter +, -, *, /: ")

        if operation == "+":
            result = a + b
        elif operation == "-":
            result = a - b
        elif operation == "*":
            result = a * b
        elif operation == "/":
            result = a / b
        else:
            print("Invalid operation")
            return

        print("Result:", result)

    except ValueError:
        print("Please enter valid numbers")
    except ZeroDivisionError:
        print("Cannot divide by zero")
```

16. JSON Exception Handling

```
import json

try:
    with open("students.json", "r") as file:
        students = json.load(file)
except FileNotFoundError:
    print("students.json does not exist")
except json.JSONDecodeError:
    print("JSON file contains invalid data")
```

JSONDecodeError occurs when the file exists but its contents are not valid JSON.

```
{
    "name": "Ahmad",
    "cgpa": 3.23,
}
```

The trailing comma makes this JSON invalid.

17. CSV Exception Handling

```
import csv

try:
    with open("students.csv", "r") as file:
        reader = csv.DictReader(file)
        for row in reader:
            print(row)
except FileNotFoundError:
    print("CSV file not found")
```

18. Debugging

Debugging means finding and fixing problems. A useful workflow is: Write → Run → Find Error → Understand Error → Fix Error → Test Again.

```
numbers = [10, 20, 30]
print(numbers[5])
```

This produces `IndexError` because index 5 does not exist. When debugging, identify the error type, line, value, and expected behavior.

19. Debugging with print()

```
marks = [80, 70, 90]

total = sum(marks)
print("Marks:", marks)
print("Total:", total)

average = total / len(marks)
print("Average:", average)
```

20. Validate Data Before Processing

```
try:
    cgpa = float(input("CGPA: "))

    if cgpa < 0 or cgpa > 4:
        print("Invalid CGPA")
    else:
        print("Valid CGPA")

except ValueError:
    print("Please enter a number")
```

21. Data Science Connection

Real datasets can have missing files, invalid numbers, empty values, incorrect data types, malformed JSON, unexpected CSV values, and division-by-zero situations. Exception handling helps data-processing programs fail safely and provide meaningful information instead of crashing.

22. Practice Tasks with Complete Solutions

Task 1: Write a Python program that divides 100 by a user-provided number. If the user enters 0, do not crash and display a meaningful message. Also handle non-numeric input.

Solution:

```
try:
    number = float(input("Enter a number: "))
    result = 100 / number
    print("Result:", result)
except ZeroDivisionError:
    print("Cannot divide by zero")
except ValueError:
    print("Invalid input")
```

```
print("Please enter a valid number")
```

Task 2: Write a Python program that asks the user to enter an integer. If the user enters invalid text, handle `ValueError` and display a clear message.

Solution:

```
try:
    number = int(input("Enter an integer: "))
    print("You entered:", number)
except ValueError:
    print("Invalid input. Please enter an integer.")
```

Task 3: Create a list of three student names. Ask for an index and safely display the student at that index. Handle an invalid index.

Solution:

```
students = ["Ahmad", "Ali", "Usman"]

try:
    index = int(input("Enter student index: "))
    print("Student:", students[index])
except ValueError:
    print("Index must be a number")
except IndexError:
    print("Invalid index")
```

Task 4: Create a student dictionary containing name and CGPA. Safely attempt to access an age key that may not exist. Handle `KeyError` and demonstrate `get()`.

Solution:

```
student = {
    "name": "Ahmad",
    "cgpa": 3.23
}

try:
    print("Age:", student["age"])
except KeyError:
    print("Age key does not exist")

print("Safe age:", student.get("age", "Not available"))
```

Task 5: Open `students.txt` in read mode. If the file does not exist, handle `FileNotFoundError` and display a useful message.

Solution:

```
try:
    with open("students.txt", "r") as file:
        data = file.read()
        print(data)
except FileNotFoundError:
    print("students.txt was not found")
```

Task 6: Read `students.json` with `json.load()`. Handle both a missing file and invalid JSON.

Solution:

```
import json

try:
    with open("students.json", "r") as file:
        students = json.load(file)
        print("Student data loaded successfully:")
        print(students)
except FileNotFoundError:
    print("students.json was not found")
except json.JSONDecodeError:
    print("students.json contains invalid JSON")
```

Task 7: Create `safe_divide(a, b)`. Return the division result, but if `b` is zero, handle `ZeroDivisionError` and return a meaningful message.

Solution:

```
def safe_divide(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        return "Cannot divide by zero"

print(safe_divide(10, 2))
print(safe_divide(10, 0))
```

Task 8: Ask the user for a CGPA. Validate that it is between 0 and 4 inclusive. Handle non-numeric input and use `raise` for an invalid CGPA.

Solution:

```
try:
    cgpa = float(input("Enter CGPA: "))

    if cgpa < 0 or cgpa > 4:
        raise ValueError("CGPA must be between 0 and 4")

    print("Valid CGPA:", cgpa)

except ValueError as error:
    print("Error:", error)
```

Task 9: Create `InvalidMarksError`. Ask for marks and raise the custom exception when marks are below 0 or above 100.

Solution:

```
class InvalidMarksError(Exception):
    pass

try:
    marks = float(input("Enter marks: "))

    if marks < 0 or marks > 100:
        raise InvalidMarksError("Marks must be between 0 and 100")

    print("Valid marks:", marks)

except ValueError:
    print("Please enter a valid number")
except InvalidMarksError as error:
    print("Error:", error)
```

Task 10: Build a Safe Student Data Analyzer that reads `students.json`, handles missing files and invalid JSON, displays students, calculates average marks, finds highest CGPA, filters CGPA ≥ 3.0 , validates marks from 0 to 100, and prevents division by zero.

Solution:

```
import json

class InvalidMarksError(Exception):
    pass

try:
    with open("students.json", "r") as file:
        students = json.load(file)

    if not students:
        print("No student data available")
    else:
        valid_students = []

        for student in students:
            marks = student.get("marks")
            cgpa = student.get("cgpa")

            if marks is None or cgpa is None:
                print("Skipping incomplete student:",
```

```

        student.get("name", "Unknown"))
        continue

    if marks < 0 or marks > 100:
        raise InvalidMarksError(
            f"Invalid marks for {student.get('name', 'Unknown')}}"
        )

    valid_students.append(student)

print("\nStudents:")
for student in valid_students:
    print(student)

if valid_students:
    average_marks = sum(
        student["marks"] for student in valid_students
    ) / len(valid_students)

    highest_cgpa = max(
        student["cgpa"] for student in valid_students
    )

    high_cgpa_students = [
        student for student in valid_students
        if student["cgpa"] >= 3.0
    ]

    print("\nAverage Marks:", average_marks)
    print("Highest CGPA:", highest_cgpa)
    print("Students with CGPA >= 3.0:")

    for student in high_cgpa_students:
        print(student["name"], student["cgpa"])
else:
    print("No valid student records available")

except FileNotFoundError:
    print("students.json was not found")
except json.JSONDecodeError:
    print("students.json contains invalid JSON")
except InvalidMarksError as error:
    print("Data validation error:", error)
except TypeError:
    print("Student data contains an unexpected data type")

```

23. Mini Project: Safe Student Data Analyzer

The mini project combines Day 8 exception handling with earlier skills: lists, dictionaries, loops, filtering, JSON, and basic data analysis.

Requirements:

1. Read students.json.
2. Handle missing file.
3. Handle invalid JSON.
4. Display students.
5. Calculate average marks.
6. Find highest CGPA.
7. Filter students with CGPA >= 3.0.
8. Validate marks.
9. Prevent division by zero.
10. Display meaningful error messages.

Sample Data

```

students = [
    {"name": "Ahmad", "marks": 85, "cgpa": 3.23},
    {"name": "Ali", "marks": 72, "cgpa": 3.10},
    {"name": "Usman", "marks": 65, "cgpa": 2.90},
    {"name": "Hamza", "marks": 91, "cgpa": 3.60}
]

```

The complete solution is provided in Task 10. Try implementing it yourself before checking the solution.

24. Quick Revision

```
try:
    # risky code
except SomeError:
    # handle error
else:
    # successful execution
finally:
    # always runs

raise ValueError("Invalid data")
```

Important exceptions: ValueError, TypeError, NameError, IndexError, KeyError, ZeroDivisionError, FileNotFoundError, JSONDecodeError.

25. Day 8 Data Science Progression

```
Python Basics
↓
Conditions + Loops
↓
Functions
↓
Lists
↓
Tuples + Sets + Dictionaries
↓
File Handling
↓
CSV + JSON
↓
Exception Handling
↓
NumPy
↓
Pandas
↓
Data Cleaning
↓
Data Visualization
↓
Statistics
↓
Machine Learning
```

26. Final Day 8 Checklist

- I understand syntax errors.
- I understand runtime errors and exceptions.
- I can use try and except.
- I can handle specific and multiple exceptions.
- I understand else and finally.
- I can use raise for validation.
- I can create custom exceptions.
- I can debug with error messages and print().
- I can safely read JSON and CSV files.
- I can build a safe student data analyzer.

DAY 8 COMPLETED: Python Exception Handling + Debugging